

Contents

What is <i>actually</i> in the COBOL extensions on Software Heritage?	1
1. Motivation & questions	1
2. Method	1
3. Experiments (settings, stated up front)	2
4. Results	2
5. Key findings & lessons (summary)	6
6. Limitations	6
7. Reproducibility & artefacts	7

What is *actually* in the COBOL extensions on Software Heritage?

An empirical study of the `.cbl` / `.CBL` file extensions in the Software Heritage (SWH) archive. Toolkit: `tools/cobol/`. Companion study on a non-programming extension: `docs/fsf_swh_study.md`.

Bottom line. A file extension is a *claim*, not a fact. Of the COBOL extension space in SWH, ~46 % is **not COBOL** (mostly one synthetic test fixture). Of the genuine COBOL, *what you conclude depends entirely on how you sample*: the typical COBOL **file** is a production program, but the typical COBOL **project** is a student exercise. Getting this right requires reading content (not extensions) and recovering provenance (which repo each file came from).

1. Motivation & questions

An extension is the cheapest possible label: `.cbl` “means” COBOL. But is it COBOL, how much of it, of what kind, and from where? We answer, for `.cbl` and `.CBL`:

- **Q1 — Contamination.** What fraction is not COBOL, and what is it instead?
- **Q2 — Composition.** Of the genuine COBOL: dialect, standard, source format, domain, maturity — and how stable are those answers under different sampling?
- **Q3 — Provenance.** Where does it come from, and how concentrated?
- **Q4 — Method.** Can a cheap deterministic layer + an LLM judge produce a reproducible characterisation we can trust?

2. Method

The same pipeline runs end-to-end and is reused across studies:

Stage	What it does
Sample	dedup by content <code>sha1_git</code> ; draw N (various frames, §3)
Fetch	raw bytes from SWH by <code>sha1_git</code> (self-verifying), cached
Indicators	deterministic metrics: LOC, divisions, EXEC SQL/CICS, COPY, COMP-3, source format
LLM-judge	enum-constrained verdict via OpenRouter (Claude Sonnet 4.6, <code>json_schema</code>): is-COBOL, dialect family, standard, source format, domain, program type, maturity
Reclassifier	zero-API content rules for the non-COBOL tail, validated against the judge
Origins	recover the repository each content came from (graph CSV + GitHub match)
Review	a web app over every label (<code>tools/cobol/review_app.py</code>) for human ground truth

The judge is asked for a single JSON object under a strict schema, so its categorical answers are drawn from fixed vocabularies (`tools/cobol/taxonomy.py`) and aggregate cleanly; a free-text **detail** carries nuance. Everything is cached and resumable (re-runs and re-judging cost zero SWH quota).

3. Experiments (settings, stated up front)

The study is a set of deliberate experiments, each **isolating one variable**. The crucial design choice is the **sampling frame** — how contents are drawn — because it is what makes numbers comparable or misleading.

#	Experiment	Sample (frame)	N judged	What it isolates -> what it shows
E1	Pilot	100, random, both extensions, no filter	46	end-to-end validation -> first contamination signal (51 % noise)
E2	Curated composition	350, random, noise-filtered (WBC_*_F00 excluded), union	317	the character of <i>filtered</i> COBOL as usually sampled by-file
E3	Population & contamination	1 000, uniform-random, no filter	526 (+474 gated)	the true file-level population + a contamination estimate
E4	Project-level view	1 000, origin-diverse (<= 1 file per repo)	896	the project-level population (no repo dominates)
E5	Cheap-classifier eval	162 (tail + synthetic + controls)	—	how well a zero-API classifier matches the LLM
E6	Provenance	full origin CSVs + GitHub matcher	—	which repositories the corpus comes from

Why three “large samples” and not one? E2, E3, E4 differ *only* in the sampling frame — filtered-by-file, uniform-by-file, one-per-repo. Comparing them **is** the experiment: it measures how much of any conclusion is an artefact of *how you drew the sample* rather than a fact about COBOL.

Fixed throughout: model `anthropic/claude-sonnet-4.6`, temperature 0, structured outputs; a cost **gate** (`n_divisions >= 2`) skips obvious non-COBOL before spending on the judge. Total ~ 2 900 judged contents, ~\$45 across all experiments.

4. Results

4.1 Contamination — the extension is a weak signal (Q1)

The single largest fact about the corpus is that **most of it is not COBOL**.

- **E1** (unfiltered pilot): 51 of 100 random files were WBC_*_F00.CBL — literally "This is cobol file number N", a *synthetic placeholder*. Corpus- wide these are **109 999 / 276 831 = ~40 %** of all deduplicated contents.
- **E3** (uniform-random 1 000, content-classified): **45.6 % non-COBOL** (95 % CI 42.5–48.7), independently **confirmed by the LLM at 47 %** (of 1 000, 526 judged real-COBOL / 474 gated; 525 of 526 judged confirmed as COBOL). The non-COBOL breaks down as 40.8 % synthetic + 2.8 % other/foreign + 1.7 % Calibre *comic-book lists* (a `.cbl` extension collision) + 0.3 % binary.

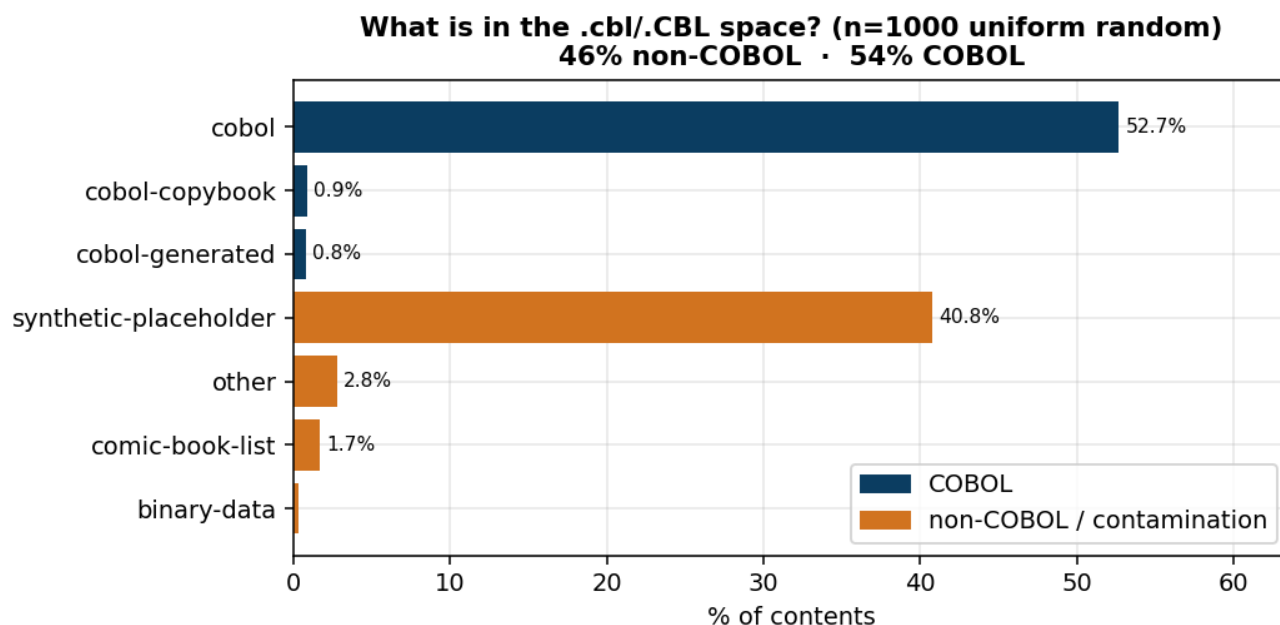


Figure 1: Contamination of the COBOL extension space

Key finding — Contamination. ~46 % of the .cbl/.CBL space is not COBOL, almost all of it a *single synthetic test fixture* (§4.3). Two independent methods (a deterministic reclassifier and the LLM) agree to within 1.4 points. **Never trust the extension; classify content.**

Casing matters too: .CBL (192 k occurrences) and .cbl (57 k) are *different populations* — a clean single-extension slice is **83 % healthcare** for .CBL vs **36 % education** for .cbl — yet the project’s mapping folds case (cf. the documented .R/.r fix). Contamination modes also differ by casing (synthetic stubs for .CBL; comic-book lists for .cbl).

4.2 The genuine COBOL — and why the sampling frame decides the story (Q2)

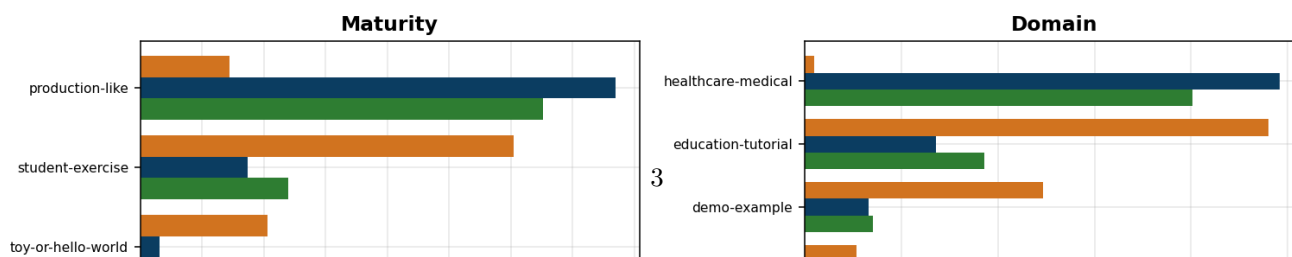
Restricting to real COBOL, one property is invariant and the rest are not.

Invariant: COBOL-85 ~98 % across *every* sampling frame. This is the one answer that does not depend on how you sample.

Everything else depends on the frame. E3 (uniform-random, file-level), E2 (filtered, file-level) and E4 (one-per-repo, project-level) give sharply different pictures:

	E3 — random (file)	E2 — filtered (file)	E4 — by repo (project)
healthcare-medical	40 %	49 %	~0 %
education-tutorial	19 %	14 %	48 %
production-like	65 %	77 %	14 %
student-exercise	24 %	17 %	60 %
median code lines	382	596	48
GnuCOBOL	65 %	70 %	41 %
COBOL-85	98 %	98 %	96 %

Sampling frame: random (file) vs curated by-file vs by-repo



Beyond that, the by-file view (E2, the “clean COBOL” people usually mean) is: GnuCOBOL ~70 %, IBM-mainframe ~11 %; fixed-format 92 %; batch 44 % / subprogram 23 % / online-CICS 18 %; copybooks (COPY) 69 %, CALL 65 %, EXEC CICS/SQL /COMP-3 all single-digit %.

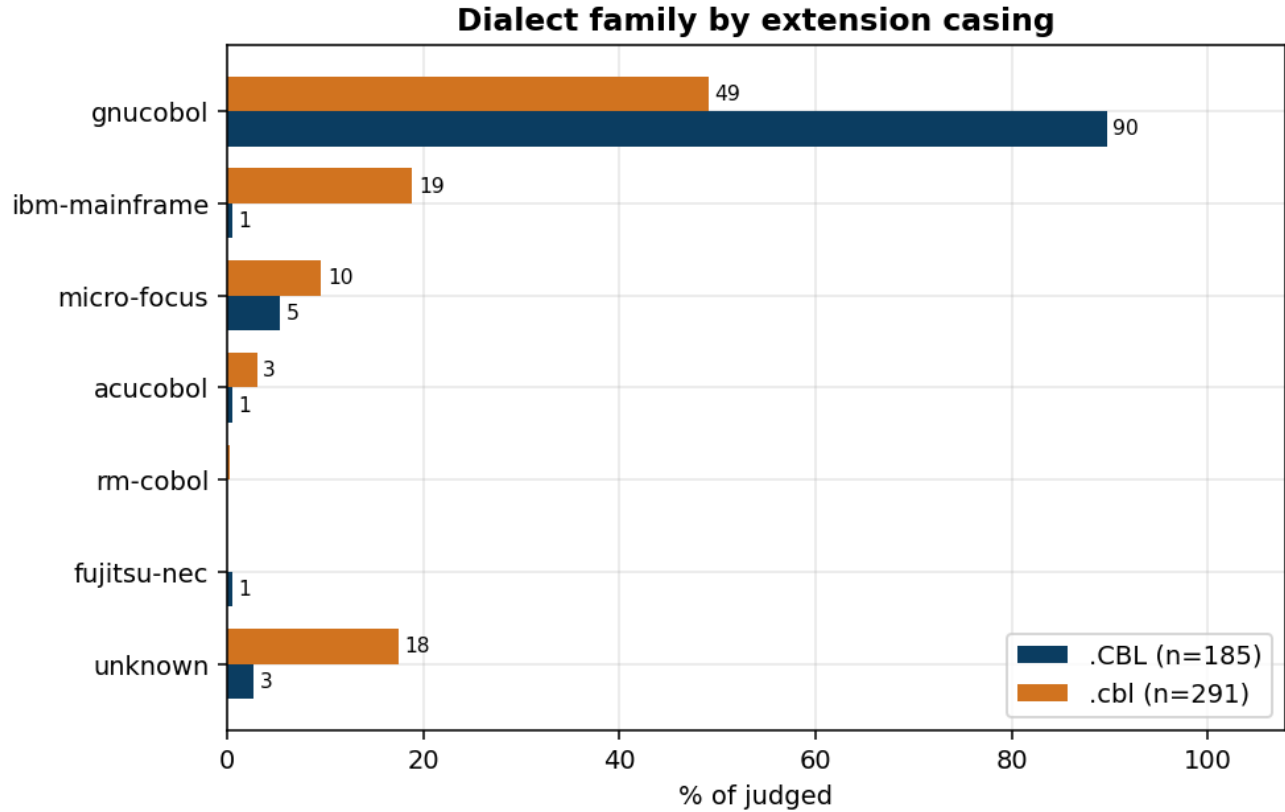


Figure 3: Dialect family by extension casing

Key finding — Public COBOL != enterprise COBOL. The archive is dominated by GnuCOBOL, COBOL-85, fixed-format code, and the mainframe idioms that motivate “COBOL modernization” (CICS, DB2/EXEC SQL, COMP-3) appear in only single-digit percentages.

4.3 Provenance — where it comes from, and how concentrated (Q3)

Origins were recovered two ways: a graph-derived CSV (authoritative, 99.7 % of lowercase, all forges) and a GitHub filename+byte-length matcher (narrow: 67 of 653, but with commit anchors). Global content dedup means the same bytes can live in many repos; the graph route usually finds the upstream.

276 600 contents from 6 278 repositories across GitHub (~143 k), GitLab (~120 k), Bitbucket, SourceForge SVN/CVS, Google Code. But the corpus is *extremely concentrated*:

Key finding — Provenance is the mechanism behind the sampling bias. A single **synthetic fixture** (fbetestpublic/repo-with-many-files-in-one-tree, a “many files in one tree” test repo) is **40 % of everything**; the **ORCA jma-receipt ecosystem** (~65 k contents) is the largest *real* system. A handful of repos dominate the file population — which is exactly why by-file sampling (§4.2) is skewed and by-repo sampling recovers the truth.

4.4 Method validation (Q4)

- **Reclassifier vs LLM (E5).** A zero-API content classifier scores **precision 1.00 / recall 0.79 / F1 0.88** against the judge on “is this COBOL” over 162 files, beating the cheap division-gate baseline (F1 0.71). It

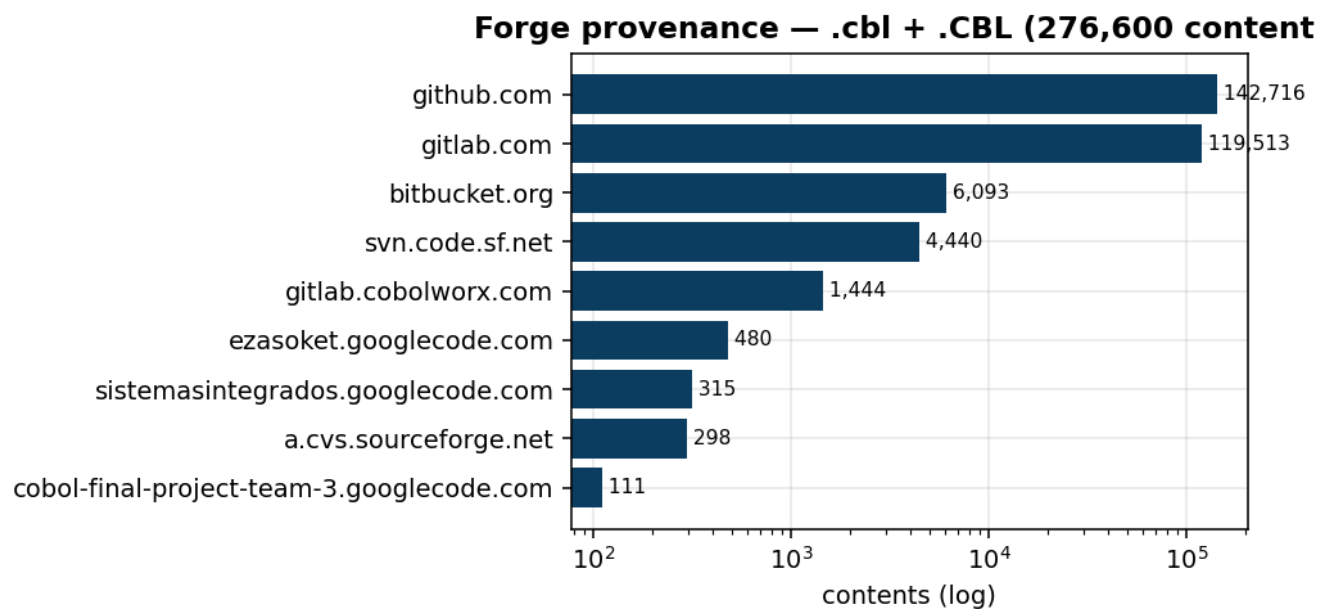


Figure 4: Forge provenance

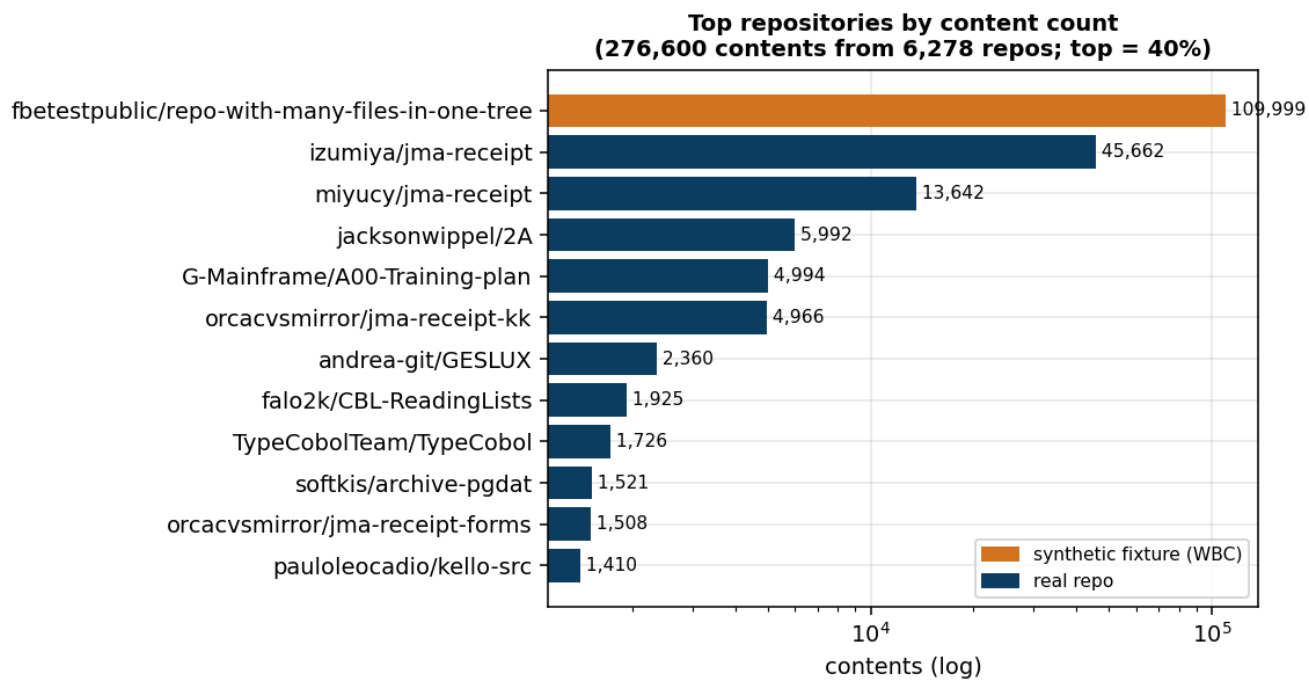


Figure 5: Top repositories by content count

recovers copybooks / OO / Japanese-comment / NUL-bearing COBOL the gate misses; the residual errors are ambiguous weak copybooks (where the LLM’s semantic reading wins).

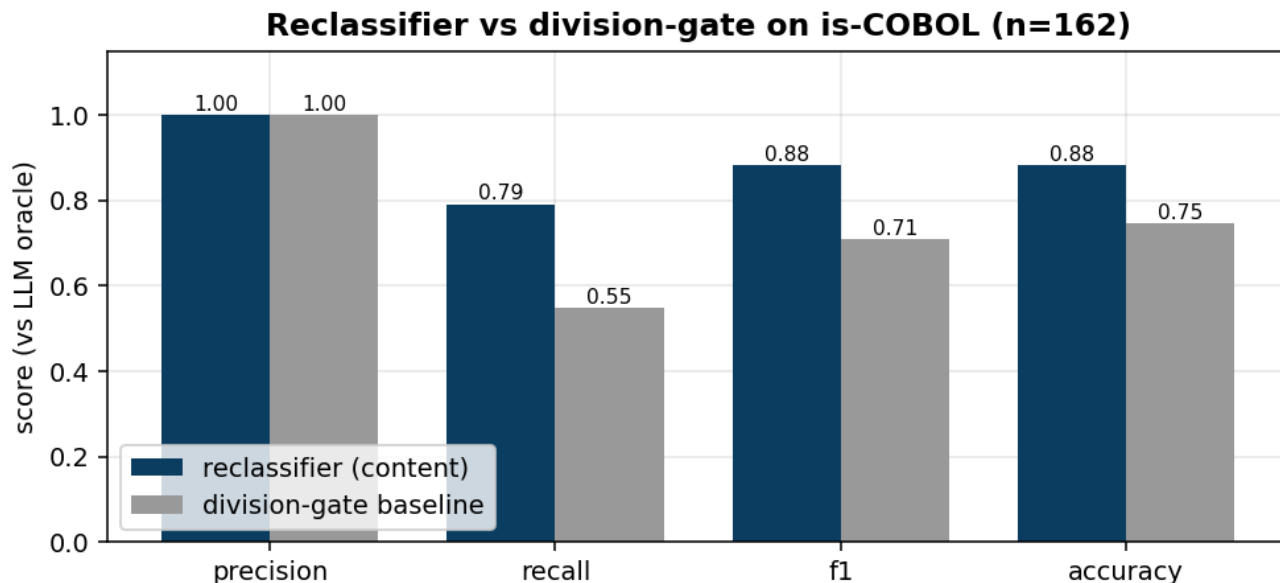


Figure 6: Reclassifier vs division-gate

- **Judge vs deterministic indicators.** The LLM’s `source_format` matches the mechanical heuristic on 95 % of judged files — an independent cross-check.
- **Judge vs reclassifier (in the review app).** They agree on is-COBOL for all 653 judged files; the 58 flagged disagreements are all the crude gate vs the content methods on the copybook/fragment boundary.

Lesson — Bootstrap a cheap classifier from the LLM, then validate it. The LLM is an expensive *oracle*; distil deterministic rules from its labels that run archive-wide for free, and *measure* their error (P/R/F1) rather than assume it. Keep the ambiguous tail with the LLM.

5. Key findings & lessons (summary)

Findings. 1. ~46 % of the COBOL extension space is **not COBOL** (mostly one synthetic fixture); casing (`.CBL!=.cbl`) indexes different populations. 2. Of genuine COBOL, **COBOL-85 (~98 %)** is the **only frame-invariant answer**; everything else depends on the sampling frame. 3. **File vs project**: typical file = production program; typical project = student exercise. 4. **Public COBOL is not enterprise COBOL** (GnuCOBOL/COBOL-85/fixed; CICS/DB2 rare). 5. Provenance is highly concentrated: one fixture = 40 %, ORCA = the largest real system.

Lessons for extension studies. 1. The extension is a weak signal — classify **content**, measure contamination first. 2. The **sampling frame** is a first-class experimental variable; report file-level *and* project-level. 3. Constrain the judge to **enums + structured outputs**; cross-check against deterministic indicators. 4. **Origins are the enabler** — they name the noise, dedup forks, and make by-repo sampling possible; recover them via the SWH **graph**, not the REST API.

6. Limitations

- **Judge accuracy is unmeasured against human ground truth** (only against a deterministic reclassifier and cross-checks). The review app (`tools/cobol/review_app.py`, with group-assertion rules) exists to build it.
- Sources > 16 000 chars are truncated for the judge (indicators use full bytes); **unknown** dialect (10–37 %, higher in the by-repo frame) reflects honest abstention on tiny snippets.

- Content-level dedup does not collapse near-duplicate forks (e.g. ORCA mirrors), so per-repo counts still over-represent mirrored systems.
- Many ORCA files carry EUC-JP/Shift-JIS Japanese comments; comment metrics are approximate on those.

7. Reproducibility & artefacts

```
python3 -m tools.cobol.sample --n 350 --seed 7 --exclude-name 'WBC.*_F00' # E2
python3 -m tools.cobol.corpus_estimate --worklist worklist_1k.csv # E3 (reclassifier sweep)
python3 -m tools.cobol.sample_diverse --n 1000 --seed 5 # E4
OPENROUTER_API_KEY=... python3 -m tools.cobol.run_study --judge --judge-min-divisions 2 --tag <name>
python3 -m tools.cobol.eval_reclassify --tail-all # E5
python3 -m tools.cobol.recover_origins # E6
python3 -m tools.cobol.make_figures ; python3 -m tools.cobol.review_app
```

Artefacts in `data/derived/cobol_study/`: `worklist*.csv`, per-content `reports/<sha>.json`, `indicators*.csv`, `summary*.{md,json}`, `origins.jsonl`, `corpus_estimate.jsonl`; figures in `docs/assets/cobol/`. Toolkit: `tools/cobol/`. Total ~ 2 900 judged contents, ~\$45, 0 parse failures.

Model: claude-sonnet-4.6 (judge) · study authored with Claude Code.